

01 文件读写与资源管理

目标：闭卷会写 `with open`、读写、复制、CSV 风格文本解析，并理解文件是否会自动关闭。

文件基本模型

文件路径

→ 打开模式

→ 获得文件对象

→ 读取或写入

→ 关闭文件

`with open` 可以自动关闭文件，优先使用。

打开模式【必须闭卷会写】

1. 会出现什么问题

文件模式选错，会导致读不了、覆盖旧内容、追加失败。

2. 最小语法

模式	含义	文件不存在
<code>r</code>	读取	报错
<code>w</code>	覆盖写	创建
<code>a</code>	追加写	创建

补充：

模式	含义	优先级
<code>rb</code>	二进制读取	会读会改
<code>wb</code>	二进制写入	会读会改
<code>r+</code>	读写	了解即可

3. 执行顺序

```
with open("data.txt", "r", encoding="utf-8") as file:
    content = file.read()
```

打开 `data.txt`
→ 返回文件对象 `file`
→ `read` 读取
→ 离开 `with` 自动关闭

4. 正常情况

文件存在，读取成功。

5. 异常情况

`r` 模式文件不存在： `FileNotFoundError`。

6. 常见错误

用 `w` 打开旧文件会清空原内容。

7. 考试可能怎么出

问 `w` 对已有文件有什么影响。

with open 【必须闭卷会写】

1. 会出现什么问题

手动 `open()` 后忘记 `close()`，资源可能泄漏；中间异常会让 `close()` 跳过。

2. 最小语法

```
with open(
    "data.txt",
    "r",
    encoding="utf-8",
) as file:
    content = file.read()
```

3. 执行顺序

进入 `with`: 打开文件
代码块中: 读写文件
离开 `with`: 自动关闭文件

4. 正常情况

读写完成后自动关闭。

5. 异常情况

`with` 块里即使抛异常，也会先关闭文件。

6. 常见错误

依赖 `__del__` 关闭文件。资源管理优先用 `with open`。

7. 考试可能怎么出

改错题：裸 `open()` 没有关闭，要求改成 `with open`。

读取方法【必须闭卷会写】

方法	返回类型	用法
<code>read()</code>	字符串	一次读完
<code>read(size)</code>	字符串	读指定字符数
<code>readline()</code>	字符串	读一行
<code>readlines()</code>	字符串列表	读所有行
<code>for line in file</code>	每次一行字符串	推荐逐行处理

最小语法

```
with open("scores.txt", "r", encoding="utf-8") as file:
    for line in file:
        line = line.strip()
        print(line)
```

常见错误

读出来的行通常带 `\n`，处理前常用 `line.strip()`。

写入方法【必须闭卷会写】

最小语法

```
with open("result.txt", "w", encoding="utf-8") as file:
    file.write("第一行\n")
    file.write("第二行\n")
```

重点

`write` 参数必须是字符串
`write` 不会自动添加换行
`writelines` 也不会自动添加换行

错误：

```
file.write(123)
```

正确：

```
file.write(str(123))
```

文件指针【必须会读会改】

1. 会出现什么问题

连续 `read()`，第二次可能为空。

2. 最小语法

```
with open("data.txt", "r", encoding="utf-8") as file:
    text1 = file.read()
    text2 = file.read()
```

第二次为空，因为第一次已经读到文件末尾。

3. 修复

```
file.seek(0)
```

`tell()` 只需识别：查看当前位置。

文件复制【必须闭卷会写】

一次性复制

```
source = "source.txt"
target = "target.txt"

with open(source, "r", encoding="utf-8") as src:
    content = src.read()

with open(target, "w", encoding="utf-8") as dst:
    dst.write(content)
```

逐行复制

```
with open(source, "r", encoding="utf-8") as src:
    with open(target, "w", encoding="utf-8") as dst:
        for line in src:
            dst.write(line)
```

图片等二进制文件用 `rb/wb`，不写 `encoding`。

CSV 风格文本【必须闭卷会写】

本专题按课程范围使用文本处理，不强制使用 `csv` 模块。

数据：

```
Tom,80
Jerry,92
```

解析：

```
line = "Tom,80"
name, score_text = line.strip().split(",")
score = int(score_text)
student = {"name": name, "score": score}
print(student)
```

读取学生并统计：

```
students = []

with open("scores.txt", "r", encoding="utf-8") as file:
    for line in file:
        name, score_text = line.strip().split(",")
        students.append({"name": name, "score": int(score_text)})

total = 0
for student in students:
    total += student["score"]

average = total / len(students)
print(average)
```

路径问题【必须会读会改】

相对路径：相对于当前工作目录
绝对路径：从盘符开始
Windows 反斜杠可能被当作转义

推荐原始字符串：

```
path = r"C:\Users\50469\data.txt"
```

pathlib 了解即可，考试优先掌握字符串路径和 with open。

文件模式速查

模式	作用	是否覆盖
r	读	不覆盖
w	写	覆盖
a	追加	不覆盖
rb	读二进制	不覆盖
wb	写二进制	覆盖

读取方法速查

```
file.read()
file.read(10)
file.readline()
file.readlines()
for line in file:
    ...
```

写入方法速查

```
file.write("文本\n")
file.writelines(["a\n", "b\n"])
```

10 个易错点

1. r 文件不存在会报错。
2. w 会覆盖旧内容。
3. a 是追加。
4. 忘记 encoding="utf-8" 可能中文乱码。
5. write() 参数必须是字符串。
6. write() 不自动换行。
7. writelines() 不自动换行。
8. read() 第二次可能为空。
9. 逐行读取要 strip()。
10. 图片复制要用 rb/wb。

8 个口头自测

1. r/w/a 的区别是什么？
2. with open 解决什么问题？

3. `read()` 返回什么？
4. `readlines()` 返回什么？
5. `for line in file` 每轮拿到什么？
6. `write(123)` 为什么错？
7. 为什么 CSV 读出的分数要 `int()`？
8. 相对路径相对哪里？

5 个闭卷文件题

1. 读取 `data.txt` 全部内容。
2. 写三行文本到 `result.txt`。
3. 追加一行日志到 `log.txt`。
4. 复制 `source.txt` 到 `target.txt`。
5. 读取 `name,score` 风格文本并统计平均分。

答案区

```
with open("source.txt", "r", encoding="utf-8") as src:
    content = src.read()

with open("target.txt", "w", encoding="utf-8") as dst:
    dst.write(content)
```

[继续学习异常处理](#)